

上下限制約付き双対単体法の実装まとめ

ENOMOTO-Solver `src/simplex.rs`

2026 年 9 月 12 日

概要

本稿は **ENOMOTO-Solver** の `src/simplex.rs` に実装されている、全変数が有限な上下限を持つ場合に特化した「有界変数双対単体法」(bounded-variable dual simplex, 関数 `solve_lp_dual/solve_lp_dual_on`) について、内部で用いられているデータ構造とその数式的な意味を対応付けながら整理する。実装は Huangfu & Hall (2015) の並列双対単体法 (dual steepest edge, bound-flipping ratio test) を下敷きにしつつ、スパース Markowitz LU 分解と Forrest–Tomlin (1972) 型の増分更新、コスト摂動によるアンチサイクリング、Bland 規則へのフォールバックを組み合わせている。

目次

1	標準形	1
2	データ構造	2
2.1	StdForm ：標準形の疎行列表現	2
2.2	Tableau ：基底とその時点の解	2
2.3	基底の三角分解：Markowitz LU + Forrest–Tomlin 更新	3
3	初期基底の構成：双対実行可能クラッシュ	3
3.1	コスト摂動によるアンチサイクリング	4
4	メインループ	4
4.1	退出行の選択 (chuzr) と双対 steepest edge	4
4.2	BTRAN と PRICE	5
4.3	進入列候補の絞り込み (chuzc1)	5
4.4	Bound-Flipping Ratio Test (BFRT) と Harris の 2 パス法	5
4.5	FTRAN, 主変数の更新, 双対 steepest edge の更新	6
4.6	基底交換と被約費用の増分更新 (update-dual)	6
4.7	基底分解の更新	7
5	終了条件	7
5.1	Bland 規則によるフォールバック	7
6	データ構造のまとめ	7

1 標準形

モデルの構造変数は $x \in \mathbb{R}^n$ ，制約は m 本で，実装上の重要な前提としてすべての構造変数が有限な上下限 $lb_j, ub_j \in \mathbb{R}$ ($-\infty, \infty$ を許さない) を持つ (`model.rs` の PyO3 境界で検証済み)。この前提により「自由変数」のケースを実装から完全に排除できる。

各制約行にスラック変数 s_i を導入し，常に等式標準形

$$Ax + \sigma \odot s = b, \quad lb \leq \begin{pmatrix} x \\ s \end{pmatrix} \leq ub$$

に帰着させる ($\sigma_i \in \{+1, -1\}$ は行の向きに依存する符号)。具体的には

$$\begin{aligned} \text{等式 } Ax = b &\Rightarrow Ax + s = b, \quad s \in [0, 0] \\ \text{不等式 } Gx \leq h &\Rightarrow Gx + s = h, \quad s \in [0, \infty) \\ \text{不等式 } Gx \geq h &\Rightarrow Gx - s = h, \quad s \in [0, \infty) \end{aligned}$$

とする。列を $z = (x, s) \in \mathbb{R}^{n_{\text{total}}}$ ($n_{\text{total}} = n + m$) とまとめ，行列 $M \in \mathbb{R}^{m \times n_{\text{total}}}$ を $M = [A \mid \Sigma]$ (Σ は対角 ± 1) とすれば，全体は

$$\min_z c^\top z \quad \text{s.t.} \quad Mz = b, \quad lb \leq z \leq ub$$

という一本の有界変数線形計画になる。すべての行が自分のスラックを基底変数として持てば $B = I$ となるため，これが初期基底として常に利用できる（後述の crash 手続きの前提）。

2 データ構造

2.1 StdForm：標準形の疎行列表現

```
struct StdForm {
    n_total: usize,      // n + m
    n_rows:  usize,      // m
    c:      Vec<f64>,     // 目的関数係数 c in R^{n_total}
    rows:   FixedRows,    // M を行方向に格納した疎行列 (CSR 相当)
    cols:   FixedRows,    // M の転置 (CSC 相当)
    b:      Vec<f64>,     // 右辺 b in R^m
    lb:     Vec<f64>,     // 下限 lb in R^{n_total}
    ub:     Vec<f64>,     // 上限 ub in R^{n_total}
}
```

数式との対応は次の通り。

$$c \leftrightarrow c, \quad M \leftrightarrow \text{rows/cols}, \quad b \leftrightarrow b, \quad (lb, ub) \leftrightarrow (lb, ub).$$

`FixedRows` は「行ごとに可変長の `Vec` を確保する」構造をやめ，全エントリを 1 本のフラットな `entries: Vec<(usize, f64)>` に連結し，行 i の担当区間を `offsets[i]..offsets[i+1]` で切り出す，一般的な CSR (Compressed Sparse Row) 形式である：

$$M_{i,\cdot} = \{(j, v) : \text{entries}[k] = (j, v), \quad k \in [\text{offsets}[i], \text{offsets}[i+1])\}.$$

`cols` は `rows` を転置した CSC 相当のビューで、一度だけ構築し、以降 `StdForm` 自体は不変 (solve 中に一切変更されない)。これにより「列 j の非零成分を求める」操作 (エントリング列の FTRAN 入力, 価格付け) が行を全走査せずに $O(\text{nnz}_j)$ で行える。

2.2 Tableau：基底とその時点の解

```
struct Tableau<'a> {
    std: &'a StdForm,
    basis: Vec<usize>,           // basis[i] = 行 i で基底の変数番号
    basis_pos: Vec<Option<usize>>, // 逆写像 basis_pos[var] = Some(i)
    nb_status: Vec<Option<NbStatus>>, // 基底なら None
    x: Vec<f64>,                 // 現在の全変数値 x in R^{n_total}
}

enum NbStatus { Lower, Upper }
```

これは古典的な単体法の「基底 B / 非基底 N 」への分割

$$\{1, \dots, n_{\text{total}}\} = \mathcal{B} \sqcup \mathcal{N}, \quad \mathcal{B} = \{\text{basis}[0], \dots, \text{basis}[m-1]\}$$

をそのまま配列で持つ実装である。`basis_pos` は $\mathcal{B}$ への所属判定と行番号引き当てを $O(1)$ にするための逆引き表であり、数学的には

$$\text{basis_pos}[j] = \begin{cases} i & j = \text{basis}[i] \ (j \in \mathcal{B}) \\ \perp & j \in \mathcal{N} \end{cases}$$

に対応する。非基底変数の状態 `NbStatus` は、有界変数単体法特有の「非基底変数はどちらの限界に張り付いているか」

$$x_j = \begin{cases} lb_j & \text{nb_status}[j] = \text{Lower} \\ ub_j & \text{nb_status}[j] = \text{Upper} \end{cases} \quad (j \in \mathcal{N})$$

を表す。構造変数は必ず両側有限なので、`NbStatus` に「自由変数」の場合分けが存在しない (doc comment 通り、これは前節の有界性前提が可能にしている)。

基底行列を $B = M_{\cdot, \mathcal{B}} \in \mathbb{R}^{m \times m}$ 、非基底部分を $N = M_{\cdot, \mathcal{N}}$ とすると、基底変数値は

$$x_{\mathcal{B}} = B^{-1}(b - Nx_{\mathcal{N}})$$

で与えられる。実装では `compute_rhs` が $rhs = b - Nx_{\mathcal{N}}$ ($O(\text{nnz}(M))$ の疎な内積の和) を計算し、`resync_basics` が三角分解を用いてこの式を解いて $x_{\mathcal{B}}$ に書き戻す。通常の反復中は $x_{\mathcal{B}}$ をこの式で毎回引き直すのではなく、ピボット操作による差分更新 (後述) で維持し、一定間隔または数値誤差検知時にのみ `recompute_basics` で「真値」に同期する。

2.3 基底の三角分解：Markowitz LU + Forrest–Tomlin 更新

B は素朴に稠密で保持せず、スパース Markowitz ピボット選択による LU 分解

$$P_r B P_c = LU$$

(L : 単位下三角, U : 上三角, P_r, P_c : 行・列置換) を `lu::factorize` で一度構築し (`sparse_lu::FtLu`), 以後のピボットでは基底の 1 列だけが変わることを利用して Forrest & Tomlin (1972) 型の階数 1 の増分更新 (`FtLu::try_update`) でこれを維持する。フルな再分解は次の 4 条件のいずれかで発火する。

1. 定期チェック (`FT_CHECK_INTERVAL=5` 反復ごと, さらに `RESIDUAL_CHECK_MULTIPLIER=20` 倍粗く) で真の基底残差 $\|A_{\mathcal{B}}x_{\mathcal{B}} - rhs\|$ が `FT_RESIDUAL_TOL` $= 10^{-4}$ を超える場合。
2. 更新後のピボット要素が `FT_MIN_PIVOT` $= 10^{-7}$ 未満になった場合 (即時)。
3. 蓄積された eta ファイルの fill が `FT_BUMP_LIMIT_FACTOR` $\times m = 64m$ を超えた場合。
4. 更新回数が `FT_MAX_UPDATES` $= 300$ を超えた場合 (無条件)。

再分解に失敗する (数値的特異) 場合は, 双対法をあきらめて主単体法 `solve_lp_on` にフォールバックする。

3 初期基底の構成：双対実行可能クラッシュ

双対単体法は「双対実行可能な初期基底」を必要とする。基底 $\mathcal{B}$ に対する単体乗数 (双対変数) を

$$y^{\top} = c_{\mathcal{B}}^{\top} B^{-1}$$

非基底 j の被約費用 (reduced cost) を

$$d_j = c_j - y^{\top} M_j$$

と定義すると, 双対実行可能性は

$$d_j \geq 0 \ (j : x_j = lb_j), \quad d_j \leq 0 \ (j : x_j = ub_j)$$

である。初期基底として全スラックを採用すると $B = I$, $c_{\mathcal{B}} = 0$ (スラックの費用は常に 0) なので $y = 0$, したがって

$$d_j = c_j \quad (\forall j).$$

よって `crash_dual_feasible` は各構造変数 j を

$$\text{nb_status}[j] = \begin{cases} \text{Lower} & c_j \geq -\varepsilon \\ \text{Upper} & c_j < -\varepsilon \end{cases} \quad (\varepsilon = \text{TOL} = 10^{-9})$$

に割り当てただけで双対実行可能性が無条件に成立する。これは両側とも有限な上下限を持つという前節の前提があって初めて常に成り立つ ($c \geq -\varepsilon$ か $c \leq \varepsilon$ のどちらかは必ず成立する)。

3.1 コスト摂動によるアンチサイクリング

主単体法側の EXPAND 法 (Gill–Murray–Saunders–Wright, 1989) に対応する退化対策として, 双対法では反復開始前に一度だけコストベクトルを摂動する (`perturb_costs`, HiGHS `HEkk::initialiseCost` と同じ設計)。

$$\text{base} = 5 \times 10^{-7} \cdot \bar{c}, \quad \bar{c} = \begin{cases} \max_j |c_j|^{1/4} & \max_j |c_j| > 100 \\ \max_j |c_j| & \text{それ以外} \end{cases}$$

(さらに「箱型」制約を持つ列が全体の 1% 未満なら $\bar{c} \leftarrow \min(\bar{c}, 1)$)。固定列 ($lb_j = ub_j$) や自由列はそのまま、片側のみに有限な列は有限でない側から遠ざかる方向へ、両側有限な列は自分の費用の符号方向へ、列インデックスから決定的に導いた擬似乱数 $r_j \in [0, 1)$ を使って

$$\Delta_j = (1 + r_j)(|c_j| + 1) \cdot \text{base}$$

だけシフトする。摂動後のコスト $\tilde{c}$ に対して `crash_dual_feasible` を実行し、反復全体は $\tilde{c}$ に基づく被約費用 d を保持する (真の目的関数 `std.c` は最適性の最終確認にのみ使う)。

4 メインループ

反復のたびに `Tableau` は基底 $\mathcal{B}$ を 1 列だけ入れ替える (p 行目が退出し、列 q が進入する)。各ステップの数式的定義を以下にまとめる。

4.1 退出行の選択 (chuzr) と双対 steepest edge

基底変数 $x_{\text{basis}[i]}$ が許容誤差 `PRIMAL_FEAS_TOL` = 10^{-7} を超えて限界を破っている行の集合を、逐次更新される `InfeasibleRows` (差分保守, 行が変化した時だけ $O(1)$ で更新) から取り出す。行 i の逸脱量

$$\delta_i = \max(lb_{\text{basis}[i]} - x_{\text{basis}[i]}, x_{\text{basis}[i]} - ub_{\text{basis}[i]}, 0)$$

に対し、双対 steepest edge 重み w_i (`DseState::w`, 初期値 $w_i = 1$, $B_0 = I$ より $\|B_0^{-\top} e_i\|^2 = 1$) を用いてスコア

$$\text{score}_i = \frac{\delta_i^2}{\max(w_i, \epsilon_{\text{floor}})}$$

を最大化する行 p を選ぶ (同点は行番号で決定的にタイブレーク)。重み w_i は

$$w_i \approx \|B^{-\top} e_i\|_2^2$$

の近似であり、通常の Dantzig 規則 (δ_i の大きさだけを見る) より、実際に 1 単位動かしたときの目的関数改善に近い量を近似する。

4.2 BTRAN と PRICE

選ばれた行 p に対し、

$$\rho_p = B^{-\top} e_p \quad (\text{すなわち } B^{\top} \rho_p = e_p \text{ を解く; BTRAN})$$

を計算し (`rho_p_buf`), 単体表のピボット行

$$a_p^{\top} = \rho_p^{\top} M \in \mathbb{R}^{n_{\text{total}}}, \quad (a_p)_j = \sum_i \rho_p(i) M_{i,j}$$

を `PRICE` で求める。 ρ_p の非零行だけを走査し、各行の疎な `std.rows.row(i)` を辿ることで、 ρ_p が疎であるほど高速になる (`a_p/touched_cols/touched` は「今回触れた列」だけを記録する差分バッファ)。

4.3 進入列候補の絞り込み (chuzc1)

退出変数が下限割れ (`leaving_infeasible_low`) か上限超過かに応じて、非基底列 j ($a_j := (a_p)_j \neq 0$) のうち、その列を動かすことで退出変数の実行可能性が回復し得る符号条件

$$\text{下限割れ: } \begin{cases} j \in \mathcal{N}_{\text{Lower}} : a_j < -\varepsilon \\ j \in \mathcal{N}_{\text{Upper}} : a_j > \varepsilon \end{cases} \quad \text{上限超過: } \begin{cases} j \in \mathcal{N}_{\text{Lower}} : a_j > \varepsilon \\ j \in \mathcal{N}_{\text{Upper}} : a_j < -\varepsilon \end{cases}$$

を満たすものを候補とし、比

$$r_j = \left| \frac{d_j}{a_j} \right|$$

を計算する (古典的な双対比検定の比)。候補が空なら、それが数値誤差の範囲内でない限り、その基底行の実行不可能性が真の一次実行不可能性の証明となり `Status::Infeasible` を返す。

4.4 Bound-Flipping Ratio Test (BFRT) と Harris の 2 パス法

候補を比 r_j の昇順 (同点は列番号) で整列した列 (実装では全ソートではなく二分ヒープ `BinaryHeap<Reverse<ChuzcCand>>` から必要な分だけ pop する) を順に見て、

$$\Delta x_j = \begin{cases} ub_j - lb_j & j \text{ が Lower} \\ -(ub_j - lb_j) & j \text{ が Upper} \end{cases}$$

だけ完全に反対側の限界へ「反転」させたときの退出変数の仮想値

$$\hat{x}_p \leftarrow \hat{x}_p - a_j \Delta x_j$$

を追跡し、目標値 `target` $\in \{lb_{\text{leaving}}, ub_{\text{leaving}}\}$ に到達 (またはオーバーシュート) する直前まで反転を確定する (幅が無限大の候補, すなわちスラックの片側無限区間はそもそも反転不可)。到達点のインデックスを `stop` とする。

数値安定性のため, `stop` で単純に確定するのではなく, 比が

$$r_{\text{stop}} - \text{HARRIS_RATIO_TOL} \leq r_j \leq r_{\text{stop}}$$

(`HARRIS\_RATIO\_TOL` = 10^{-7}) の窓に入る候補 ($j \leq \text{stop}$ の中から後ろ向きにのみ探索) のうち $|a_j|$ が最大のものを真の進入列 q として選び直す (Harris 1973 の 2 パス法に類似, `HiGHS chooseFinalLargeAlpha` を参考に, ただし過大な逸脱を避けるため前方には探索しない)。stop より手前の候補はすべて完全反転として確定させ, これらの反転効果は 1 回の追加 FTRAN

$$B \alpha_F = \sum_{j:\text{反転}} M_j \Delta x_j$$

としてまとめて基底変数へ反映する (`ftran-bfirt`)。

4.5 FTRAN, 主変数の更新, 双対 steepest edge の更新

進入列 q の方向ベクトル, および DSE の交差項を

$$\alpha = B^{-1} M_q, \quad \tau = B^{-1} \rho_p$$

として FTRAN で求める (`alpha_buf`, `tau_buf`)。主変数のステップ幅は、反転処理後の退出変数値 x_{leaving} を用いて

$$\theta_q = \frac{x_{\text{leaving}} - \text{target}}{\alpha_p}$$

で決まり、全基底変数を

$$x_{\text{basis}[i]} \leftarrow x_{\text{basis}[i]} - \alpha_i \theta_q \quad (\forall i), \quad x_q \leftarrow x_q + \theta_q$$

と更新する ($\alpha_p \theta_q = x_{\text{leaving}} - \text{target}$ より、行 p はちょうど目標限界に一致する)。

DSE 重みは Forrest–Goldfarb 型の階数 1 更新 (この実装では双対版として再導出済み)

$$\rho_i := \frac{\alpha_i}{\alpha_p} \quad (i \neq p), \quad w_i \leftarrow \max(w_i - 2\rho_i \tau_i + \rho_i^2 w_p, \varepsilon_{\text{floor}}), \quad w_p \leftarrow \max\left(\frac{w_p}{\alpha_p^2}, \varepsilon_{\text{floor}}\right)$$

で維持する (`DseState::update_after_pivot`)。

4.6 基底交換と被約費用の増分更新 (update-dual)

基底配列を更新する：

$$\text{nb_status}[\text{leaving}] \leftarrow \begin{cases} \text{Lower} \\ \text{Upper} \end{cases}, \quad \text{basis}[p] \leftarrow q, \quad \text{nb_status}[q] \leftarrow \perp.$$

被約費用は BTRAN をやり直さず、ピボット行 a_p を使って全列を一括更新する (Sherman–Morrison による階数 1 更新から c_B の交差項が相殺して導かれる closed form)：

$$\theta_d = \frac{d_q}{\alpha_p}, \quad d_j \leftarrow d_j - \theta_d (a_p)_j \quad (\forall j).$$

これにより、次回以降の反復でも d は「その時点の基底に対する真の被約費用」であり続け、BTRAN + 内積で再計算する必要がない。

4.7 基底分解の更新

最後に Forrest–Tomlin の階数 1 更新 `lu.try_update(p, a_enter, FT_MIN_PIVOT)` を試み、ピボットが小さすぎる ($< 10^{-7}$) などの理由で失敗すれば、その場でフル再分解を行う。

5 終了条件

- 最適：`chuzr` が実行不能行を一つも見つけれられない、すなわち摂動後コスト $\tilde{c}$ について一次実行可能かつ双対実行可能。このとき、真のコスト `std.c` に対して被約費用を計算し直し (`fresh_d`) ,

$$d_j \geq -\varepsilon \quad (j \in \mathcal{N}_{\text{Lower}}), \quad d_j \leq \varepsilon \quad (j \in \mathcal{N}_{\text{Upper}})$$

が真に成り立てば `Status::Optimal`。摂動が真の双対非実行可能性を覆い隠していた場合 (稀) は、現在の基底が一次実行可能であることを利用して主単体法フェーズ 2 に切り替えて仕上げる。

- 一次実行不可能：chuzc1 が候補を一つも見つけれられず、かつその行の逸脱が丸め誤差の範囲を超えている場合（行のスケールに応じた相対許容誤差で判定）。または BFRT の反転が全候補を使い切っても目標限界に届かない場合。
- 反復上限：MAX_ITEERS= 20,000 に達した場合はベストエフォートの解を返す。

5.1 Bland 規則によるフォールバック

$\theta_q d_q$ （このピボットの目的関数への実際の寄与）が $\text{STALL_PROGRESS_EPS} = 10^{-9}$ を $\text{stall_limit} = \max(5m, 500)$ 回連続で下回ると、コスト摂動と Harris 法だけでは退化サイクルを避けられなかったと判断し、以降は Bland (1977) の最小添字規則に切り替える (bland_mode)：退出行はインデックス最小の実行不能な基底変数、進入列は数値的に許容できる中で最小インデックスの候補（全滅時のみ許容度を無視）。Bland 規則は理論的に有限収束が保証される唯一の規則であり、一度 ON になったら解が求まるまで戻らない。

6 データ構造のまとめ

Rust 構造体／フィールド	数式上の対象	備考
StdForm{c, rows, cols, b, lb, ub}	c, M, M^T, b, lb, ub	CSR/CSC 相当の FixedRows, solve 中不変
Tableau{basis, basis_pos, nb_status, x}	B, B^{-1} 写像, $\mathcal{N}$ の限界, z	反復ごとに差分更新
NbStatus::{Lower, Upper}	非基底変数の張り付き先	自由変数のケースなし
sparse_lu::FtLu	$B = P_r^{-1} L U P_c^{-1} + \text{eta}$ 更新列	Markowitz + Forrest-Tomlin
DseState::w	$w_i \approx \ B^{-T} e_i\ ^2$	双対 steepest edge 重み
d: Vec<f64>	$d_j = c_j - y^T M_j$	毎反復 closed form で増分更新
a_p / touched_cols	$a_p^T = \rho_p^T M$ の疎表現	PRICE の出力, 差分クリア
InfeasibleRows{rows, pos}	$\{i : x_{\text{basis}(i)} \text{ が実行不能}\}$	増分保守された集合
ChuzcCand{j, a_pj, dj, ratio}	BFRT 候補 (j, a_j, d_j, r_j)	二分ヒープで昇順に取り出す

7 まとめ

この実装は、教科書的な有界変数双対単体法 (BTRAN $\rightarrow$ PRICE $\rightarrow$ chuzr $\rightarrow$ chuzc $\rightarrow$ FTRAN $\rightarrow$ 基底交換) を骨格としながら、(1) 全構造変数が有限上下限を持つという ENOMOTO-Solver 固有の前提を利用して双対実行可能な初期基底を無条件に構成できるようにし、(2) 被約費用と双対 steepest edge 重みを both closed-form の階数 1 更新で保守することで BTRAN の再計算を反復ごとに避け、(3) BFRT と Harris 型 2 パス法・コスト摂動・Bland 規則という 3 段構えの退化・数値安定性対策を備え、(4) 基底の実体は Markowitz ピボットによるスパース LU と Forrest-Tomlin

増分更新で保持する，という構成になっている。